

SQL Code

```
1  CREATE DATABASE retail_store_mp;
2  USE retail_store_mp;
3
4  CREATE TABLE customers (
5      customer_id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100) NOT NULL,
6      email VARCHAR(100) NOT NULL UNIQUE,
7      phone VARCHAR(15),
8      created_at DATETIME DEFAULT CURRENT_TIMESTAMP
9  );
10
11 CREATE TABLE products (
12     product_id INT PRIMARY KEY AUTO_INCREMENT,
13     name VARCHAR(100) NOT NULL,
14     category VARCHAR(50) NOT NULL,
15     price DECIMAL(10,2) NOT NULL,
16     stock_quantity INT NOT NULL DEFAULT 0,
17     added_on DATETIME DEFAULT CURRENT_TIMESTAMP
18 );
19
20 CREATE TABLE orders (
21     order_id INT PRIMARY KEY AUTO_INCREMENT,
22     customer_id INT,
23     order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
24     status VARCHAR(20) DEFAULT 'Pending',
25     total_amount DECIMAL(10,2),
26     FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
27 );
28
29 CREATE TABLE order_items (
30     order_item_id INT PRIMARY KEY AUTO_INCREMENT,
31     order_id INT,
32     product_id INT,
33     quantity INT NOT NULL CHECK (quantity > 0),
34     item_price DECIMAL(10,2) NOT NULL,
35     FOREIGN KEY (order_id) REFERENCES orders(order_id),
36     FOREIGN KEY (product_id) REFERENCES products(product_id)
37 );
38
39 CREATE TABLE payments (
40     payment_id INT PRIMARY KEY AUTO_INCREMENT,
41     order_id INT,
42     payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
43     amount_paid DECIMAL(10,2) NOT NULL CHECK (amount_paid > 0),
44     method VARCHAR(20) NOT NULL,
45     FOREIGN KEY (order_id) REFERENCES orders(order_id)
46 );
47
48 CREATE TABLE product_reviews (
49     review_id INT PRIMARY KEY AUTO_INCREMENT,
50     product_id INT,
```

```
51         customer_id INT,
52         rating INT NOT NULL,
53         review_text TEXT,
54         review_date DATETIME DEFAULT CURRENT_TIMESTAMP,
55         FOREIGN KEY (product_id) REFERENCES products(product_id),
56         FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
57     );
58 SELECT *
59 FROM CUSTOMERS;
60
61 # LEVEL 01---BASIC----->
62 # 1. Retrieve customer names and emails for email marketing--->
63
64 SELECT NAME,EMAIL
65 FROM CUSTOMERS;
66
67 # 2. View complete product catalog with all available details--->
68
69 SELECT *
70 FROM PRODUCTS;
71
72 # 3. List all unique product categories--->
73
74 SELECT DISTINCT CATEGORY
75 FROM PRODUCTS;
76
77 # 4. Show all products priced above ₹1,000--->
78
79 SELECT *
80 FROM PRODUCTS
81 WHERE PRICE > 1000;
82
83 # 5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)--->
84
85 SELECT *
86 FROM PRODUCTS
87 WHERE PRICE BETWEEN 2000 AND 5000;
88
89 # 6. Fetch data for specific customer IDs (e.g., from loyalty program list)--->
90
91 WITH CUSTOMER_ORDER_COUNTS AS (
92     SELECT
93         CUSTOMER_ID,
94         COUNT(ORDER_ID) AS TOTAL_ORDERS
95     FROM ORDERS
96     GROUP BY CUSTOMER_ID
97 ),
98 LOYALTY_CATEGORIZATION AS (
99     SELECT
100         CUSTOMER_ID,
101         TOTAL_ORDERS,
102         CASE
103             WHEN TOTAL_ORDERS >= 15 THEN 'LOYAL'
104             WHEN TOTAL_ORDERS >= 7 THEN 'POTENTIAL LOYAL'
```

```
105         WHEN TOTAL_ORDERS >= 3 THEN 'HIGH SPENDER'
106         ELSE 'RECENT JOINEES'
107     END AS LOYALTY_PROGRAM
108 FROM CUSTOMER_ORDER_COUNTS
109 )
110 SELECT
111     CUSTOMER_ID,
112     TOTAL_ORDERS,
113     LOYALTY_PROGRAM
114 FROM LOYALTY_CATEGORIZATION;
115
116 # 7. Identify customers whose names start with the letter 'A'--->
117
118 SELECT NAME , CUSTOMER_ID
119 FROM CUSTOMERS
120 WHERE NAME LIKE "A%";
121
122 # 8. List electronics products priced under ₹3,000--->
123
124 SELECT *
125 FROM PRODUCTS
126 WHERE CATEGORY = "ELECTRONICS" AND PRICE < 3000;
127
128 # 9. Display product names and prices in descending order of price--->
129
130 SELECT NAME , PRICE
131 FROM PRODUCTS
132 ORDER BY PRICE DESC;
133
134 # 10. Display product names and prices, sorted by price and then by name--->
135
136 SELECT NAME , PRICE
137 FROM PRODUCTS
138 ORDER BY PRICE ASC , NAME ASC;
139
140 # LEVEL 02---FILTERING AND FORMATTING----->
141 # 1. Retrieve orders where customer information is missing (possibly due to data
142 migration or deletion)--->
143
144 SELECT
145     O.ORDER_ID,
146     O.ORDER_DATE,
147     O.TOTAL_AMOUNT
148 FROM
149     ORDERS O
150 JOIN
151     CUSTOMERS C
152 ON
153     O.CUSTOMER_ID = C.CUSTOMER_ID
154 WHERE
155     O.CUSTOMER_ID IS NULL
156     OR TRIM(O.CUSTOMER_ID) = ''
157     OR C.CUSTOMER_ID IS NULL;
```

```
158 # 2. Display customer names and emails using column aliases for frontend readability--  
159 ->  
160 SELECT  
161     NAME AS CUSTOMER_NAME,  
162     EMAIL AS CUSTOMER_EMAIL  
163 FROM  
164     CUSTOMERS;  
165  
166 # 3. Calculate total value per item ordered by multiplying quantity and item price--->  
167  
168 SELECT  
169     ORDER_ITEM_ID,  
170     SUM(QUANTITY * ITEM_PRICE) AS TOTAL_VALUE  
171 FROM  
172     ORDER_ITEMS  
173 GROUP BY  
174     ORDER_ITEM_ID;  
175  
176 # 4. Combine customer name and phone number in a single column--->  
177  
178 SELECT  
179     CONCAT(NAME , "---->" , PHONE) AS CUSTOMER_DETAILS  
180 FROM  
181     CUSTOMERS;  
182  
183 # 5. Extract only the date part from order timestamps for date-wise reporting--->  
184  
185 SELECT  
186     DATE(ORDER_DATE) AS CUSTOMER_ORDER_DATE,  
187     COUNT(*) AS TOTAL_ORDER  
188 FROM  
189     ORDERS  
190 GROUP BY  
191     DATE(ORDER_DATE)  
192 ORDER BY  
193     CUSTOMER_ORDER_DATE DESC;  
194  
195 # 6. List products that do not have any stock left--->  
196  
197 SELECT  
198     PRODUCT_ID,  
199     NAME,  
200     STOCK_QUANTITY  
201 FROM  
202     PRODUCTS  
203 WHERE  
204     STOCK_QUANTITY = 0;  
205  
206 # LEVEL 03---AGGREGATION----->  
207 # 1. Count the total number of orders placed--->  
208  
209 SELECT COUNT(*) AS TOTAL_ORDER  
210 FROM ORDERS;
```

```
211
212 # 2. Calculate the total revenue collected from all orders--->
213
214 SELECT SUM(AMOUNT_PAID) AS TOTAL_REVENUE
215 FROM PAYMENTS;
216
217 # 3. Calculate the average order value--->
218
219 SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE
220 FROM ORDERS;
221
222 # 4. Count the number of customers who have placed at least one order--->
223
224 SELECT
225     COUNT(DISTINCT CUSTOMER_ID) AS NUMBER_OF_CUSTOMER
226 FROM
227     ORDERS;
228
229 # 5. Find the number of orders placed by each customer--->
230
231 SELECT
232     COUNT(DISTINCT ORDER_ID) AS NUMBER_OF_ORDERS , CUSTOMER_ID
233 FROM
234     ORDERS
235 GROUP BY
236     CUSTOMER_ID;
237
238 # 6. Find total sales amount made by each customer--->
239
240 SELECT
241     DISTINCT SUM(TOTAL_AMOUNT) AS SALES_AMOUNT_PER_CUSTOMER , CUSTOMER_ID
242 FROM
243     ORDERS
244 GROUP BY
245     CUSTOMER_ID;
246
247 # 7. List the number of products sold per category--->
248
249 SELECT
250     COUNT(DISTINCT PRODUCT_ID) , CATEGORY
251 FROM
252     PRODUCTS
253 GROUP BY
254     CATEGORY;
255
256 # 8. Find the average item price per category--->
257
258 SELECT
259     ROUND(AVG(PRICE),2) , CATEGORY
260 FROM
261     PRODUCTS
262 GROUP BY
263     CATEGORY;
264
```

```
265 # 9. Show number of orders placed per day--->
266
267 SELECT
268     COUNT(DISTINCT ORDER_ID) AS NUMBER_OF_ORDER , ORDER_DATE
269 FROM
270     ORDERS
271 GROUP BY
272     ORDER_DATE
273 ORDER BY
274     ORDER_DATE DESC;
275
276 # 10. List total payments received per payment method--->
277
278 SELECT
279     SUM(DISTINCT AMOUNT_PAID) AS TOTAL_PAYMENTS , METHOD
280 FROM
281     PAYMENTS
282 GROUP BY
283     METHOD;
284
285 # LEVEL 04---MULTI-TABLE QUERRIES (JOINS)----->
286 # 1. Retrieve order details along with the customer name (INNER JOIN)--->
287
288 SELECT
289     O.ORDER_ID,
290     O.ORDER_DATE,
291     O.TOTAL_AMOUNT,
292     O.STATUS,
293     C.NAME
294 FROM
295     ORDERS O
296 JOIN
297     CUSTOMERS C
298 ON
299     O.CUSTOMER_ID = C.CUSTOMER_ID;
300
301 # 2. Get list of products that have been sold (INNER JOIN with order_items)--->
302
303 SELECT
304     DISTINCT P.PRODUCT_ID,
305     P.NAME
306 FROM
307     PRODUCTS P
308 JOIN
309     ORDER_ITEMS OI
310 ON
311     P.PRODUCT_ID = OI.PRODUCT_ID;
312
313 # 3. List all orders with their payment method (INNER JOIN)--->
314
315 SELECT
316     DISTINCT O.ORDER_ID,
317     P.METHOD
318 FROM
```

```
319         ORDERS O
320 JOIN
321     PAYMENTS P
322 ON
323     O.ORDER_ID = P.ORDER_ID;
324
325 # 4. Get list of customers and their orders (LEFT JOIN)--->
326
327 SELECT
328     C.CUSTOMER_ID,
329     C.NAME,
330     O.ORDER_ID,
331     O.ORDER_DATE
332 FROM
333     CUSTOMERS C
334 LEFT JOIN
335     ORDERS O
336 ON
337     C.CUSTOMER_ID = O.CUSTOMER_ID;
338
339 # 5. List all products along with order item quantity (LEFT JOIN)--->
340
341 SELECT
342     DISTINCT P.PRODUCT_ID,
343     P.NAME,
344     OII.QUANTITY
345 FROM
346     PRODUCTS P
347 LEFT JOIN
348     ORDER_ITEMS OII
349 ON
350     P.PRODUCT_ID = OII.PRODUCT_ID;
351
352 # 6. List all payments including those with no matching orders (RIGHT JOIN)--->
353
354 SELECT
355     P.PAYMENT_ID,
356     P.PAYMENT_DATE,
357     P.AMOUNT_PAID,
358     O.ORDER_ID,
359     O.ORDER_DATE
360 FROM
361     ORDERS O
362 RIGHT JOIN
363     PAYMENTS P
364 ON
365     O.ORDER_ID = P.ORDER_ID;
366
367 # 7. Combine data from three tables: customer, order, and payment--->
368
369 SELECT
370     C.CUSTOMER_ID,
371     C.NAME,
372     O.ORDER_ID,
```

```
373         O.ORDER_DATE,
374         P.PAYMENT_ID,
375         P.PAYMENT_DATE,
376         P.AMOUNT_PAID
377     FROM
378     CUSTOMERS C
379     JOIN
380     ORDERS O
381     ON
382     C.CUSTOMER_ID = O.CUSTOMER_ID
383     JOIN
384     PAYMENTS P
385     ON
386     O.ORDER_ID = P.ORDER_ID;
387
388     # LEVEL 05---SUBQUERIES(INNER QUERIES)----->
389     # 1. List all products priced above the average product price--->
390
391     SELECT
392     PRODUCT_ID , NAME , PRICE
393     FROM
394     PRODUCTS
395     WHERE
396     PRICE > (
397         SELECT AVG(PRICE)
398         FROM PRODUCTS
399     )
400     ORDER BY PRICE DESC;
401
402     # 2. Find customers who have placed at least one order--->
403
404     SELECT
405     CUSTOMER_ID , NAME
406     FROM
407     CUSTOMERS
408     WHERE
409     CUSTOMER_ID
410     IN (
411         SELECT CUSTOMER_ID
412         FROM ORDERS
413     );
414
415     # 3. Show orders whose total amount is above the average for that customer--->
416
417     SELECT
418     O1.ORDER_ID , O1.CUSTOMER_ID , O1.TOTAL_AMOUNT
419     FROM
420     ORDERS O1
421     WHERE
422     O1.TOTAL_AMOUNT > (
423         SELECT AVG(O2.TOTAL_AMOUNT)
424         FROM ORDERS O2
425         WHERE O2.CUSTOMER_ID = O1.CUSTOMER_ID
426     );
```



```
427
428 # 4. Display customers who haven't placed any orders--->
429
430 SELECT
431     CUSTOMER_ID , NAME
432 FROM
433     CUSTOMERS
434 WHERE
435     CUSTOMER_ID
436 NOT IN (
437     SELECT CUSTOMER_ID
438     FROM ORDERS
439     );
440
441 # 5. Show products that were never ordered--->
442
443 SELECT
444     PRODUCT_ID , NAME
445 FROM
446     PRODUCTS
447 WHERE
448     PRODUCT_ID
449 NOT IN (
450     SELECT PRODUCT_ID
451     FROM ORDER_ITEMS
452     WHERE PRODUCT_ID IS NOT NULL
453     );
454
455 # 6. Show highest value order per customer--->
456
457 SELECT
458     O.CUSTOMER_ID , O.ORDER_ID , O.TOTAL_AMOUNT
459 FROM
460     ORDERS O
461 JOIN (
462     SELECT CUSTOMER_ID , MAX(TOTAL_AMOUNT) AS MAX_ORDER_VALUE
463     FROM ORDERS
464     GROUP BY CUSTOMER_ID
465     ) MO
466 ON O.CUSTOMER_ID = MO.CUSTOMER_ID
467 AND O.TOTAL_AMOUNT = MO.MAX_ORDER_VALUE;
468
469 # 7. Highest Order Per Customer (Including Names)--->
470
471 SELECT
472     C.NAME,
473     O.ORDER_ID,
474     O.TOTAL_AMOUNT
475 FROM
476     ORDERS O
477 JOIN (
478     SELECT CUSTOMER_ID , MAX(TOTAL_AMOUNT) AS MAX_ORDERVALUE
479     FROM ORDERS
480     GROUP BY CUSTOMER_ID
```

```
481         ) MO
482     ON    O.CUSTOMER_ID = MO.CUSTOMER_ID
483     AND    O.TOTAL_AMOUNT = MO.MAX_ORDERVALUE
484     JOIN    CUSTOMERS C
485     ON    O.CUSTOMER_ID = C.CUSTOMER_ID;
486
487     # LEVEL 06---SET OPERATIONS----->
488     # 1. List all customers who have either placed an order or written a product review---
489     >
490     SELECT
491         CUSTOMER_ID , NAME , EMAIL
492     FROM
493         CUSTOMERS
494     WHERE CUSTOMER_ID IN (
495         SELECT CUSTOMER_ID
496         FROM ORDERS
497         UNION
498         SELECT CUSTOMER_ID
499         FROM PRODUCT_REVIEWS
500     );
501
502     # 2. List all customers who have placed an order as well as reviewed a product
503     [intersect not supported]--->
504     SELECT
505         C.CUSTOMER_ID,
506         C.NAME,
507         C.EMAIL
508     FROM CUSTOMERS C
509     WHERE CUSTOMER_ID IN (
510         SELECT CUSTOMER_ID
511         FROM (
512             SELECT DISTINCT CUSTOMER_ID
513             FROM ORDERS
514             UNION ALL
515             SELECT DISTINCT CUSTOMER_ID
516             FROM PRODUCT_REVIEWS
517         ) AS COMBINED
518     GROUP BY CUSTOMER_ID
519     HAVING COUNT(*) = 2
520     );
```